

### Week 3 - Design Write-Up

#### Data:

First, the `demand\_enriched\_corrupted.parquet` file could be split from before and after January 16, 2026 to form a historical baseline with over 6 million rows and then our multiple weeks before February 1, 2026. From the historical batch, I began by first calculating statistical range metrics for all the columns: min, max, mean, standard deviation, 95<sup>th</sup> percentile and 99<sup>th</sup> percentile values for anything datatype with integer or float values. Then from the historical data, I also calculated null rates for all data categories, especially for binary categories, to know if extra holidays were being flagged. Finally, I calculated data drift statistics namely Population Stability Indices and Kolmogorov-Smirnov statistics comparing the clean historical data to our data of interest. Finally, I tested for duplication of rows attending to the time stamps. From this initial work each of these metrics provided some insight into corrupted data.

#### Issues found + Impact:

1. First, the range statistics provided help with data corruption in the `trip\_counts`. Historically, 310 trips was the maximum number of trips in a given day. By giving a tolerance of 150% of that value or any negative values, I was able to isolate sentinel numbers: -5 (in 60 rows), -1 (in 75 rows), 9999 (in 61 rows), and 99999 (in 50 rows). Together that is 246 obviously invalid rows, spread across 56 distinct zones, on every of the 17 days. So I dropped anything with an extreme value that would throw off any capacity for calculating normal ranges.
2. There were also many mislabeled `is\_holiday` flags. I cross-referenced the `is\_holiday` column against the actual 2026 US Federal holiday calendar. The data correctly flags January 1 (New Year's Day), January 19 (MLK Day), and February 16 (Presidents Day). But the dataset flagged 1/7 - 1/21 as well which is fourteen consecutive days. Inside the slide-defined Jan 16 - Feb 1 batch window, five of those dates fall. Across all of 2026 the mislabel total is 76,608 rows, or 23.0% of all 2026 data. The model uses `is\_holiday` as a feature so mislabeling shifts predicted demand toward holiday-shaped patterns for two normal week so we cannot get meaningful signal out of this.
3. Also there were many duplicated (PULocationID, time\_bucket) keys. Within the slide 1/16 - 2/1/26, I found zero duplicates. However, my validator scans the entire incoming batch and when run against the full corrupted parquet it found 10,085 duplicate keys in 2/7-2/28/26. The validator's behavior is the right defensive design: real bad-data batches do not respect arbitrary date cutoffs, and the validator needs to catch duplicates the moment they appear. The runtime fix keeps the first occurrence per (zone, time) key and drops later duplicates.
4. And finally, the requested `lag\_1week` contamination I could not solve but want to document here. Three zones: 161, 162, and 186 had lag\_1week column with data from

different zones. I tried a few statistical methods on the old data, but no method registered a false positive rate below 28%. The best was a Pearson correlation test but I believe that this comes from small 17-day samples that will have difficulty naturally determining variance. Looking on to week 4, I think this task is better suited for data drift rather than data validation.

### Validation schedule choice and justification

At present the validation schedule occurs every hour as the silent failure is my greatest fear in the deployment of this system. Accordingly, every hour the CI component of the graceful degradation strategy (detailed below is run) to ensure that the system is up and running. Moreover, whenever the week3/validation or /backend folders are updated, the system also reruns to ensure any updated pushed code has been successfully implemented. This update schedule ensures that a real issue is detected within an hour or immediately when uploaded and I favor more frequent checks for less downtime.

### Graceful degradation strategy

After building out a demand-forecasting API from week 2, the biggest goal is to catch the data corruption in the paraquets of data provided. From the course, there were few types of ways of responding to errors and I wanted to make sure I was prepared for Type II errors primarily. That is to say, if the system degrades, I want to catch it and not fail silently. So critical issues will stop deployment to the CI, and ambiguous errors would either stop or be flagged immediately and produce a public facing issue on the GitHub repo.

To do this, I have three layers to catch these failures:

1. In the CI pipeline, GitHub actions serve as the primary notification as it runs each hour or when I push new code. I check out the course repo to make sure no significant changes have been made and to ensure the parquet file does not live in my solution but instead is properly referenced by the file management system of GitHub so that I am not re-duplicating the large files.
2. In the API deployment I have my second layer. The loaded parquet is cleaned (via `clean_dataframe()`) which applies the previously mentioned three checks and fixes any inline flags. These correspond to the first three data quality errors identified above (1. Dropping rows where `'trip_count'` is a sentinel number, 2. Deduplicate anything with `'PULocationID'` or `'timebucket'` already specified, 3. Reverts mislabeled `'is_holiday'` categories.
3. A planned and pre-staged layer for future weeks that, during runtime, checks inputs into publicly exposed API and validates quality inputs to ensure to not waste runtime and compute.

From these three layers we have: CI check which makes sure problems are not with my deployment, then correctly predicts over recent data, and then finally ensures queries are

correctly challenged if they are bad. For the gracefulness of the failure, every fix is logged with a warning ensuring that I have no silent failures. But if over 5% of the inputs are dropped, then the second layer raises a challenge (`'DataLoadTooBadError'`) which says that so much of the data is problematic, the service needs to be investigated. This is a “loud-failure” mode wherein I’ve decided that over 5% corruption of the data suggests fundamental problems with the inputs.

This was validated in the 1/16-2/1/26 data batch where 246 rows were dropped for a rate of .26% well within the tolerance of the 5% ceiling. Thereby, I have a first pass justification of this working.